

Sistemas Operativos

1er Recu 2° Parcial 2C2025 – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría - Single choice: elija para cada pregunta la opción correcta (solo una es válida)

- 1- Algunos procesos no podrán ejecutar (b)
- 2- Ninguna es correcta (d)
- 3- Más grande que la RAM (b)
- 4- Permisos unix (c)
- 5- Locks (c)
- 6- Ninguna es correcta (d)
- 7- La cantidad máxima de archivos que se podrían crear (a)

Teoría - Desarrollar: responda las siguientes preguntas justificando sus respuestas

1-

Paginación: no hay políticas de ubicación, dado que es indistinto el frame utilizado para guardar una página.

Segmentación:

- First fit: se elige la primera partición libre donde quepa el segmento, comenzando a buscar desde el primer hueco libre.
- Next fit: similar a First Fit, pero comenzando desde el último lugar de ubicación
- Best fit: se elige la partición libre de menor tamaño, donde quepa el segmento
- Worst fit: se elige la partición libre de mayor tamaño, donde quepa el segmento

2 -

Lista no exhaustiva de pasos realizados:

- El proceso realiza una syscall para solicitar apertura de archivo
- El SO valida la existencia del archivo y permisos necesarios para realizar la operación
- El archivo es abierto, creando entradas en la lista de archivos abiertos por proceso, y en la lista global
- El proceso realiza una syscall para solicitar la escritura de un byte al final del archivo
- El SO busca un bloque libre nuevo y lo asigna al archivo (actualizando estructuras para manejo del espacio libre, y el FCB)
- Se realiza la escritura en el nuevo bloque asignado.

3 -

- a) **Verdadero.** Dado que el sistema de carga y descarga de bloques del archivo a/desde ram no es otro que el mecanismo de memoria virtual del SO, el mismo tratará los bloques como páginas, por lo que es más eficiente que ambos tengan el mismo tamaño.
- b) **Verdadero.** Si ocurriera trashing, tendríamos varios fallos de página en ese período de tiempo, los cuales implicarían cada uno una operación de I/O para traer de disco la página solicitada

Práctica - Desarrollar: resuelva los siguientes ejercicios justificando sus respuestas

1-

Formato de las direcciones lógicas:

- Offset: 10 bits (por indicar máximo de fragmentación interna de 1023)
- Página: 7 bits (por el máximo de 128 páginas por proceso)
- Segmento: 3 bits (20 - 17)

Referencias:

- 135145: seg: 1, pág: 3, offset: 1001 => dirección y permisos válidos => **frame: 10, offset: 1001**
- 405707: seg: 3, pag: 12, offset: 203 => **dirección invalida** (el segmento 3 tiene solo 12285/1024 = 11,9... = 12 páginas (desde la 0 hasta la 11))
- 270340: seg: 2, pág: 8, offset: 4 => dirección y permisos válidos => **frame: 35, offset: 4**
- 6032: seg: 0, pag: 5, offset: 912 => dirección válida, **permisos inválidos** (el segmento 0 no se puede ejecutar)

2-

- a) Dado que las primeras dos referencias no generan un reemplazo y el proceso ya tenía tres páginas cargadas, podemos asumir que el sistema tiene un máximo de 5 páginas para el proceso. El hecho de que existan frames libres indica que la asignación es fija y el alcance es local.
- b) Alcanza con que el proceso disponga de 6 páginas para que, si siempre pidiera la página ausente de memoria, se produzca un fallo de página, retrasando la ejecución del proceso. Este comportamiento indicaría la ausencia del principio de localidad.

3-

Como el máximo teórico del FS es de 12 TB y los bloques de 3072 bytes => $12 \text{ TB} / 3072 \text{ bytes} = 2^{32} \text{ bytes} \Rightarrow$ punteros de 4 bytes.

Con bloques de 3072 bytes y punteros de 4 bytes, tenemos 768 punteros por bloque.

Archivos de hasta 6100 bytes deberían implicar la menor cantidad de accesos a disco posibles: tenemos que asegurarnos que no se usen bloques de punteros, por lo que necesitaremos dos punteros directos en el inodo.

La lectura individual del byte 7000 no debe implicar más de dos accesos a disco: respetando el requisito anterior, tenemos que pensar en que el 3er puntero del inodo sea una indirección simple (no podría ser un puntero directo porque no llegaríamos al requisito de almacenar 1730 MB).

Ninguna lectura de un byte individual debe implicar más de 3 accesos a disco: me impide tener punteros indirectos triples.

Conformación final:

Tipo puntero	Bytes totales direccionados
Puntero directo	3072
Puntero directo	6144
Puntero indirecto simple	2365440
Puntero indirecto doble	1814304768 (1730 MB)

4-

Datos:

- Disco 512 GiB
- Volúmen 1: FAT 16 TMR = 2 GiB Tabla FAT ocupa 128 KiB
- Volúmen 2: FAT 32
- Tam. Cluster Vol1 = Tam. Cluster Vol2

a) Los punteros en FAT16 son de 16 bits = 2 Bytes

Sabiendo que la Tabla FAT ocupa 128 KiB

$$128 \text{ KiB} = \text{Tam. Puntero} \times \text{Cant. Punteros}$$

$$128 \text{ KiB} = 2 \text{ Bytes} \times \text{Cant. Punteros}$$

$$2^{17} / 2^1 = \text{Cant. Punteros} = 2^{16} \text{ Punteros} \rightarrow \text{Utiliza todos los punteros de FAT16}$$

Como el TMR del Vol1 es 2 GiB = 2^{31}

$$2^{31} = \text{Tam. Cluster} \times \text{Cant. Punteros}$$

$$2^{31} = \text{Tam. Cluster} \times 2^{16}$$

$$2^{31} / 2^{16} = \text{Tam. Cluster} = 2^{15} = 32 \text{ KiB}$$

b) TMT del Vol2 = Tam. Cluster x Cant. Punteros, Por ser FAT32 $\rightarrow 2^{28}$ Punteros

$$\text{TMT} = 32 \text{ KiB} \times 2^{28} \text{ Punteros} = 2^{15} \text{ Bytes} \times 2^{28} = 2^{43} = 8 \text{ TiB}$$

Como en el disco sólo me quedan 510 GiB disponibles (512 GiB - 2 GiB), entonces el **TMR = 510 GiB**

c) Al poder direccionar el volumen 2 sólo hasta 510 GiB, significa que utiliza menos entradas de las que dispone el formato, por ende:

Para direccionar al menos 510 GiB con clusters de 32 KiB, requiero al menos 2^{24} Entradas en la tabla fat:

$$\text{Tam. Cluster} \times \text{Cant. Punteros} = 2^{15} \times 2^{24} = 2^{39}$$

$$512 \text{ GiB} \geq 510 \text{ GiB}$$

$$\text{Tam. Tabla Fat} = \text{Tam. Puntero} \times \text{Cant. Punteros}$$

$$\text{Tam. Tabla Fat} = 4 \text{ Bytes} \times 2^{24} \text{ Punteros}$$

$$\text{Tam. Tabla Fat} = 2^2 \text{ Bytes} \times 2^{24} \text{ Punteros} = 2^{26} = 64 \text{ MiB}$$

d) Siendo que el volúmen 2 con FAT32 no ocupa el total de direcciones disponibles, podemos mejorar este uso y reducir la fragmentación interna máxima por archivo **reduciendo el tamaño de cluster**, de esta manera podríamos ocupar todos los punteros disponibles, pudiendo incluso crear más archivos y reducir la fragmentación interna.

Para el volumen 1 efectuar el mismo cambio representa por un lado reducir la fragmentación interna pero por otro lado reduce el tamaño máximo direccionable del filesystem.

5-

Se debe considerar que los cerrojos son **sugeridos**, de manera que el proceso 4 podrá operar sobre el archivo aun con un cerrojo sobre el mismo. Aparte, dado que son compartidos, varios procesos podrán leer al mismo tiempo, pero solo uno podrá escribir a la vez (cuando se tome el lock).

Nota: Esta solución considera que las syscalls sobre archivos no son bloqueantes.

- El Proceso 1 ejecuta primero, imprimiendo 'B' y volviendo a ready. Lista de bloqueados: vacía
- El proceso 2 ejecuta, bloqueando en "lock" por querer tomar un lock para escritura. No imprime nada. Lista de bloqueados: P2
- El proceso 3 ejecuta, pudiendo tomar el lock aún cuando el proceso 1 no lo ha liberado, por lo tanto imprime 'B', volviendo a Ready. Lista de bloqueados: P2
- El proceso 4 ejecuta, sin tomar lock, intenta leer y lo logra dado que el lock existente es sugerido. Imprime 'B'. Lista de bloqueados: P2
- El proceso 1 libera el lock
- El proceso 3 libera el lock
- El proceso 2 logra adquirir el lock, modifica el archivo en imprime 'A'.

Impresiones finales: P1: 'B', P3: 'B', P4: 'B', P2: 'A'